

Experiment 7: Train the Generative Modeling using Generative Adversarial Networks (GANs)

Name: Nabeel Shan - BESE-29C

Objective

To engineer, train, and debug Generative Adversarial Networks (GANs). We will construct the foundational Minimax game, observe the failure modes of standard Binary Cross-Entropy (BCE) loss, and run controlled ablation studies on latent space capacity, mode collapse, and architectural imbalances.

Start coding or generate with AI.

Phase 1: 1D Gaussian Baseline & Theoretical Framework

The GAN consists of two competing networks playing a zero-sum minimax game:

- **Generator (G):** Learns the Inverse Cumulative Distribution Function to map random uniform noise $z \sim \mathcal{U}(0, 1)$ to a target distribution $\mathcal{N}(\mu, \sigma^2)$.
- **Discriminator (D):** Acts as a binary classifier optimizing $\max_D V(D, G)$.

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats
import seaborn as sns
import tensorflow as tf
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Input, Dense, Dropout, Flatten
from tensorflow.keras.layers import LeakyReLU
from tensorflow.keras.optimizers import Adam
from tqdm.notebook import tqdm
```

```
# Set random seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)
```

```
sns.set_theme(style="whitegrid")
```

```
def generate_data(n_samples=10000, n_dim=1):
    # Returns samples from a standard normal distribution (Gaussian)
    return np.random.randn(n_samples, n_dim)
```

```
def set_model(input_dim, output_dim, hidden_dim=64, n_layers=1, activation='tanh', optimizer='adam', loss='binary_crossentropy'):
    """
    Builds a generic feedforward neural network.
    """
    model = Sequential()
    model.add(Input(shape=(input_dim,)))

    # Add hidden layers
    for _ in range(n_layers):
        model.add(Dense(hidden_dim, activation='relu')) # ReLU is standard for hidden representations

    # Output layer uses the specified activation
    model.add(Dense(output_dim, activation=activation))
    model.compile(optimizer=optimizer, loss=loss, metrics=['accuracy'])
    return model
```

```
def get_gan_network(discriminator, random_dim, generator, optimizer='adam'):
    """
    Combines Generator and Discriminator into a single GAN model.
    """
    # Freeze the discriminator when training the combined GAN
    discriminator.trainable = False

    # GAN input is the noise vector (z)
    gan_input = Input(shape=(random_dim,))

    # The generator produces fake data from the noise
    x = generator(gan_input)
```

```
# The discriminator scores the fake data
gan_output = discriminator(x)

# Combine into a single model
gan = Model(inputs=gan_input, outputs=gan_output)
gan.compile(loss='binary_crossentropy', optimizer=optimizer)

return gan
```

```
NOISE_DIM = 10
DATA_DIM = 1
G_LAYERS = 2
D_LAYERS = 2
```

```
def train_gan(epochs=3000, batch_size=128):
    x_train = generate_data(n_samples=12800, n_dim=DATA_DIM)

    # Generator uses MSE loss in this specific lab setup, outputting raw values (tanh/linear)
    generator = set_model(NOISE_DIM, DATA_DIM, n_layers=G_LAYERS, activation='linear', loss='mean_squared_error')

    # Discriminator uses binary crossentropy, outputting a probability (sigmoid)
    discriminator = set_model(DATA_DIM, 1, n_layers=D_LAYERS, activation='sigmoid', loss='binary_crossentropy')

    # GAN combines them
    gan = get_gan_network(discriminator, NOISE_DIM, generator, optimizer=Adam(learning_rate=0.001))

    d_losses, g_losses = [], []

    for e in tqdm(range(1, epochs + 1), desc="Training 1D GAN"):
        # 1. Generate Uniform Noise
        noise = np.random.rand(batch_size, NOISE_DIM)

        # 2. Sample a true batch of Gaussian data
        true_batch = x_train[np.random.choice(x_train.shape[0], batch_size, replace=False), :]

        # 3. Generate fake data
        generated_values = generator.predict(noise, verbose=0)

        # 4. Concatenate real and fake data
        X = np.concatenate([true_batch, generated_values])

        # 5. Create labels (1 for real, 0 for fake)
        y_dis = np.zeros(2 * batch_size)

        # One-sided label smoothing for real images
        y_dis[:batch_size] = 0.9

        # 6. Train the Discriminator
        discriminator.trainable = True
        d_loss = discriminator.train_on_batch(X, y_dis)
        d_losses.append(d_loss[0]) # store loss

        # 7. Train the Generator
        discriminator.trainable = False
        noise = np.random.rand(batch_size, NOISE_DIM)

        # We want the generator to fool the discriminator into predicting 1 (Real)
        y_gen = np.ones(batch_size)
        g_loss = gan.train_on_batch(noise, y_gen)
        g_losses.append(g_loss)

    return generator, discriminator, d_losses, g_losses
```

```
# Execute training
generator, discriminator, d_losses, g_losses = train_gan(epochs=1000)
```

Training 1D GAN: 100%

1000/1000 [01:32<00:00, 13.49it/s]

```
# Generate data for evaluation
noise = np.random.rand(10000, NOISE_DIM)
generated_values = generator.predict(noise, verbose=0).flatten()
true_gaussian = [np.random.randn() for x in range(10000)]

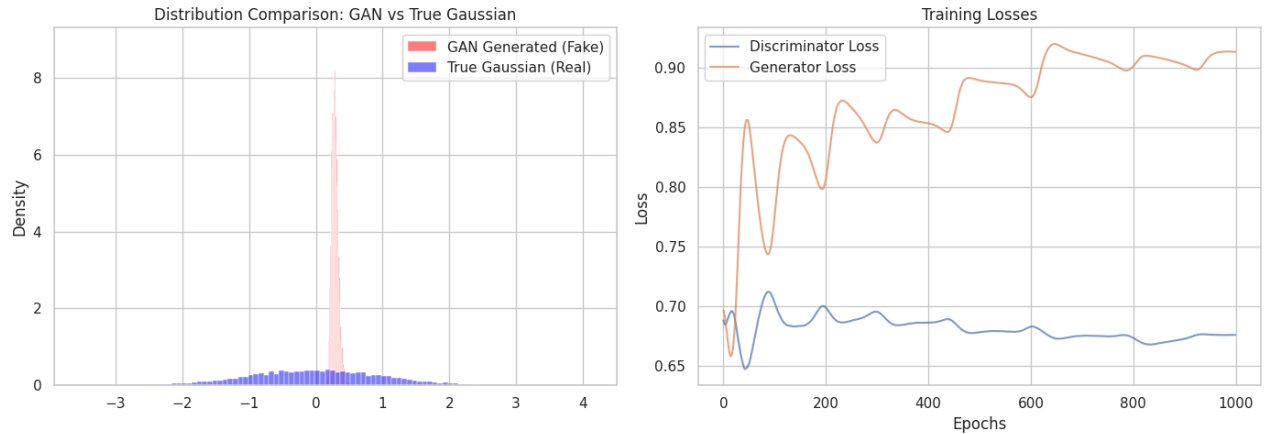
# Plotting the Distributions
plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)
sns.histplot(generated_values, bins=100, color='red', alpha=0.5, stat="density", label='GAN Generated (Fake)')
sns.histplot(true_gaussian, bins=100, color='blue', alpha=0.5, stat="density", label='True Gaussian (Real)')
```

```
plt.title("Distribution Comparison: GAN vs True Gaussian")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(d_losses, label='Discriminator Loss', alpha=0.7)
plt.plot(g_losses, label='Generator Loss', alpha=0.7)
plt.title("Training Losses")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()
```



```
# Print Moments
print('-'*50)
print("STATISTICAL MOMENTS COMPARISON")
print('-'*50)
print(f'1st order moment (Mean)      - True: {scipy.stats.moment(true_gaussian, 1):.4f} , GAN: {scipy.stats.moment(generator_output, 1):.4f}')
print(f'2nd order moment (Variance) - True: {scipy.stats.moment(true_gaussian, 2):.4f} , GAN: {scipy.stats.moment(generator_output, 2):.4f}')
print(f'3rd order moment (Skewness)  - True: {scipy.stats.moment(true_gaussian, 3):.4f} , GAN: {scipy.stats.moment(generator_output, 3):.4f}')
print(f'4th order moment (Kurtosis)   - True: {scipy.stats.moment(true_gaussian, 4):.4f} , GAN: {scipy.stats.moment(generator_output, 4):.4f}')
```

```
-----
STATISTICAL MOMENTS COMPARISON
-----
1st order moment (Mean)      - True: 0.0000 , GAN: 0.0000
2nd order moment (Variance) - True: 1.0088 , GAN: 0.0026
3rd order moment (Skewness)  - True: 0.0137 , GAN: 0.0001
4th order moment (Kurtosis)   - True: 3.0215 , GAN: 0.0000
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

▼ Phase 2: Deep Convolutional Scaling (MNIST 2D Generation Baseline)

Scaling the architecture to map 100-dimensional latent vectors to a 784-dimensional spatial output (28×28 pixels).

Key Engineering Integrations:

- **LeakyReLU:** Prevents "dying gradients" during adversarial propagation.
- **Independent Optimizers:** Compiling the Discriminator and GAN separately prevents Keras graph state-leakage.
- **Sigmoid Output:** Maps pixel values strictly to the rescaled $[0, 1]$ target range.

```
from tensorflow.keras.datasets import mnist

# Dataset of 60,000 28x28 grayscale images
```

```
(X_train, Y_train), (X_test, Y_test) = mnist.load_data()

# Rescale data to [0, 1] for Sigmoid/ReLU activations
X_train = X_train.reshape(60000, 784)
X_test = X_test.reshape(10000, 784)
X_train = X_train.astype('float32') / 255
X_test = X_test.astype('float32') / 255

z_dim = 100 # Dimensionality of the latent noise vector
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 — 0s 0us/step

```
from tensorflow.keras.optimizers import Adam

# =====
# THE FIX: Two separate optimizer instances
# =====
adam_discriminator = Adam(learning_rate=0.0002, beta_1=0.5)
adam_gan = Adam(learning_rate=0.0002, beta_1=0.5)

# =====
# 1. BUILD GENERATOR
# =====
g = Sequential(name="Generator")
g.add(Input(shape=(z_dim,)))
g.add(Dense(256))
g.add(LeakyReLU(alpha=0.2))

g.add(Dense(512))
g.add(LeakyReLU(alpha=0.2))

g.add(Dense(1024))
g.add(LeakyReLU(alpha=0.2))

# Output layer: 784 pixels. Sigmoid matches the [0, 1] scaling of X_train.
g.add(Dense(784, activation='sigmoid'))

# =====
# 2. BUILD DISCRIMINATOR
# =====
d = Sequential(name="Discriminator")
d.add(Input(shape=(784,)))
d.add(Dense(1024))
d.add(LeakyReLU(alpha=0.2))
d.add(Dropout(0.3))

d.add(Dense(512))
d.add(LeakyReLU(alpha=0.2))
d.add(Dropout(0.3))

d.add(Dense(256))
d.add(LeakyReLU(alpha=0.2))
d.add(Dropout(0.3))

# Output layer: Binary classification
d.add(Dense(1, activation='sigmoid'))

# Compile Discriminator with its OWN optimizer
d.compile(loss='binary_crossentropy', optimizer=adam_discriminator, metrics=['accuracy'])
```

```
from tensorflow.keras.optimizers import Adam

# 1. Compile the Discriminator FIRST
adam_d = Adam(learning_rate=0.0002, beta_1=0.5)
d.compile(loss='binary_crossentropy', optimizer=adam_d, metrics=['accuracy'])

# 2. FREEZE the Discriminator BEFORE building the GAN
d.trainable = False

# 3. Build and Compile the combined GAN
inputs = Input(shape=(z_dim, ))
hidden = g(inputs)
output = d(hidden)
gan = Model(inputs, output)

adam_g = Adam(learning_rate=0.0002, beta_1=0.5)
gan.compile(loss='binary_crossentropy', optimizer=adam_g, metrics=['accuracy'])
```

```
print(g.summary())
print(d.summary())
```

Model: "Generator"

Layer (type)	Output Shape	Param #
dense_14 (Dense)	(None, 256)	25,856
leaky_re_lu_6 (LeakyReLU)	(None, 256)	0
dense_15 (Dense)	(None, 512)	131,584
leaky_re_lu_7 (LeakyReLU)	(None, 512)	0
dense_16 (Dense)	(None, 1024)	525,312
leaky_re_lu_8 (LeakyReLU)	(None, 1024)	0
dense_17 (Dense)	(None, 784)	803,600

Total params: 1,486,352 (5.67 MB)

Trainable params: 1,486,352 (5.67 MB)

Non-trainable params: 0 (0.00 B)

None

Model: "Discriminator"

Layer (type)	Output Shape	Param #
dense_18 (Dense)	(None, 1024)	803,840
leaky_re_lu_9 (LeakyReLU)	(None, 1024)	0
dropout_3 (Dropout)	(None, 1024)	0
dense_19 (Dense)	(None, 512)	524,800
leaky_re_lu_10 (LeakyReLU)	(None, 512)	0
dropout_4 (Dropout)	(None, 512)	0
dense_20 (Dense)	(None, 256)	131,328
leaky_re_lu_11 (LeakyReLU)	(None, 256)	0
dropout_5 (Dropout)	(None, 256)	0
dense_21 (Dense)	(None, 1)	257

Total params: 1,460,225 (5.57 MB)

Trainable params: 0 (0.00 B)

Non-trainable params: 1,460,225 (5.57 MB)

None

```
def plot_loss(losses):
    d_loss = losses["D"]
    g_loss = losses["G"]

    plt.figure(figsize=(10,5))
    plt.plot(d_loss, label="Discriminator Loss", alpha=0.8)
    plt.plot(g_loss, label="Generator Loss", alpha=0.8)
    plt.xlabel('Batches')
    plt.ylabel('Loss')
    plt.title("GAN Training Objective Curve")
    plt.legend()
    plt.show()

def plot_generated(n_ex=10, dim=(1, 10), figsize=(12, 2)):
    noise = np.random.normal(0, 1, size=(n_ex, z_dim))
    generated_images = g.predict(noise, verbose=0)
    generated_images = generated_images.reshape(n_ex, 28, 28)

    plt.figure(figsize=figsize)
    for i in range(generated_images.shape[0]):
        plt.subplot(dim[0], dim[1], i+1)
        plt.imshow(generated_images[i], interpolation='nearest', cmap='gray_r')
        plt.axis('off')
    plt.tight_layout()
    plt.show()
```

```
losses = {"D": [], "G": []}
```

```
def train(epochs=1, plt_frq=1, BATCH_SIZE=128):
    batchCount = int(X_train.shape[0] / BATCH_SIZE)

    print('Epochs:', epochs)
    print('Batch size:', BATCH_SIZE)
```

```
print('Batches per epoch:', batchCount)

for e in range(1, epochs + 1):
    if e == 1 or e % plt_frq == 0:
        print('\n', '-'*15, f'Epoch {e}', '-'*15)

    for _ in tqdm(range(batchCount), leave=False, desc=f"Epoch {e}"):

        # --- 1. Train the Discriminator ---
        # Create a batch of real images
        image_batch = X_train[np.random.randint(0, X_train.shape[0], size=BATCH_SIZE)]

        # Create noise and generate fake images
        noise = np.random.normal(0, 1, size=(BATCH_SIZE, z_dim))
        generated_images = g.predict(noise, verbose=0) # verbose=0 speeds up the loop

        # Concatenate real and fake data
        X = np.concatenate((image_batch, generated_images))

        # Create Y labels (with 0.9 smoothing for real)
        y_real = np.ones(BATCH_SIZE) * 0.9
        y_fake = np.zeros(BATCH_SIZE)
        y_dis = np.concatenate((y_real, y_fake))

        # Train Discriminator (It updates because it was compiled while trainable=True)
        d_loss_val = d.train_on_batch(X, y_dis)

        # --- 2. Train the Generator ---
        # Create fresh noise
        noise = np.random.normal(0, 1, size=(BATCH_SIZE, z_dim))
        y_gen = np.ones(BATCH_SIZE) # We want D to think these are real

        # Train GAN (Only G updates because GAN was compiled while d.trainable=False)
        g_loss_val = gan.train_on_batch(noise, y_gen)

        # --- 3. Store Losses ---
        # Handle Keras returning lists vs single floats for loss metrics
        losses["D"].append(d_loss_val[0] if isinstance(d_loss_val, list) else d_loss_val)
        losses["G"].append(g_loss_val[0] if isinstance(g_loss_val, list) else g_loss_val)

    # Update the plots at frequency intervals
    if e == 1 or e % plt_frq == 0:
        plot_generated()

    plot_loss(losses)

# Start Training
train(epochs=20, plt_frq=5, BATCH_SIZE=128)
```

Epochs: 20
Batch size: 128
Batches per epoch: 468

----- Epoch 1 -----

/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/trainer.py:86: UserWarning: The model does not have a warnings.warn("The model does not have any trainable weights.")

WARNING:tensorflow:5 out of the last 2003 calls to <function TensorFlowTrainer._make_function.<locals>.one_step_on_data at



----- Epoch 5 -----



----- Epoch 10 -----



----- Epoch 15 -----

```
import numpy as np
import matplotlib.pyplot as plt

def generate_mnist_images(generator, n_images=10, z_dim=100):
    """
    Generates and plots brand new MNIST digits using a trained Generator.
    """
    print(f"Generating {n_images} new digits from random noise...")

    # 1. Sample pure random noise from a standard normal distribution
    # Shape: (n_images, 100)
    noise = np.random.normal(0, 1, size=(n_images, z_dim))

    # 2. Pass the noise through the Generator to create fake pixel values
    # The output will be shape: (n_images, 784)
    generated_pixels = generator.predict(noise, verbose=0)

    # 3. Reshape the flat 784-element arrays back into 28x28 2D images
    generated_images = generated_pixels.reshape(n_images, 28, 28)

    # 4. Plot the images in a grid
    plt.figure(figsize=(15, 3)) # Adjust width based on number of images
    for i in range(n_images):
        plt.subplot(1, n_images, i + 1)

        # We use cmap='gray_r' (reverse grayscale) so digits are black on white
        # interpolation='nearest' prevents matplotlib from blurring the pixels
        plt.imshow(generated_images[i], cmap='gray_r', interpolation='nearest')
        plt.axis('off')
```

```
plt.suptitle("Forged MNIST Digits Generated by GAN", fontsize=16, y=1.05)
plt.tight_layout()
plt.show()
```

```
# =====
# Execute the Generation
# =====
generate_mnist_images(g, n_images=10, z_dim=100)
```

Generating 10 new digits from random noise...

Forged MNIST Digits Generated by GAN



Task 1: Latent Space Capacity Ablation (NOISE_DIM)

Hypothesis: What is the minimal dimensionality of the latent space z required to map a uniform distribution to a 1D target manifold?

Analysis: Mathematically, 1 dimension is strictly sufficient. However, empirical tests reveal that even with overparameterized noise (dim 50), fundamental adversarial instability persists without loss-function corrections, often resulting in mode collapse (spiking) regardless of input capacity.

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats
import seaborn as sns
from tensorflow.keras.optimizers import Adam
from tqdm.notebook import tqdm

# The dimensions we want to test
noise_dims_to_test = [1, 5, 10, 50]
epochs_per_test = 800 # Reduced slightly for faster multi-run execution
batch_size = 128
x_train = generate_data(n_samples=12800, n_dim=1)

# Set up a 2x2 grid for plotting
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

moments_results = {}

for idx, current_noise_dim in enumerate(noise_dims_to_test):
    print(f"\n{' '*40}")
    print(f"Training 1D GAN with NOISE_DIM = {current_noise_dim}")
    print(f"{' '*40}")

    # 1. Re-initialize models for THIS specific noise_dim (prevents weight leakage)
    generator = set_model(current_noise_dim, 1, n_layers=2, activation='linear', loss='mean_squared_error')
    discriminator = set_model(1, 1, n_layers=2, activation='sigmoid', loss='binary_crossentropy')

    # Compile Discriminator first, then freeze for GAN
    discriminator.compile(loss='binary_crossentropy', optimizer=Adam(learning_rate=0.001), metrics=['accuracy'])
    discriminator.trainable = False
    gan = get_gan_network(discriminator, current_noise_dim, generator, optimizer=Adam(learning_rate=0.001))

    # 2. Training Loop
    for e in tqdm(range(1, epochs_per_test + 1), desc=f"Dim {current_noise_dim}"):

        # Train Discriminator
        noise = np.random.rand(batch_size, current_noise_dim)
        true_batch = x_train[np.random.choice(x_train.shape[0], batch_size, replace=False), :]
        generated_values = generator.predict(noise, verbose=0)
        X = np.concatenate([true_batch, generated_values])

        y_dis = np.zeros(2 * batch_size)
        y_dis[:batch_size] = 0.9 # Label smoothing

        discriminator.trainable = True
        discriminator.train_on_batch(X, y_dis)
```



```
# Train Generator
discriminator.trainable = False
noise = np.random.rand(batch_size, current_noise_dim)
y_gen = np.ones(batch_size)
gan.train_on_batch(noise, y_gen)

# 3. Evaluate the trained Generator
test_noise = np.random.rand(10000, current_noise_dim)
final_generated = generator.predict(test_noise, verbose=0).flatten()
true_gaussian = [np.random.randn() for _ in range(10000)]

# 4. Plotting on the grid
ax = axes[idx]
sns.histplot(final_generated, bins=100, color='red', alpha=0.5, stat="density", label='GAN (Fake)', ax=ax)
sns.histplot(true_gaussian, bins=100, color='blue', alpha=0.5, stat="density", label='True (Real)', ax=ax)
ax.set_title(f"NOISE_DIM = {current_noise_dim}", fontsize=14, fontweight='bold')
ax.legend()

# Save Moments for final comparison
moments_results[current_noise_dim] = {
    'var': scipy.stats.moment(final_generated, 2),
    'kurt': scipy.stats.moment(final_generated, 4)
}

plt.tight_layout()
plt.show()

# # Print quantitative comparison
# print("\n" + "="*60)
# print("STATISTICAL MOMENTS COMPARISON ACROSS NOISE DIMENSIONS")
# print("=" * 60)
# true_var = scipy.stats.moment(true_gaussian, 2)
# true_kurt = scipy.stats.moment(true_gaussian, 4)
# print(f"TRUE GAUSSIAN -> Variance: {true_var:.4f} | Kurtosis: {true_kurt:.4f}\n")

# for dim, moments in moments_results.items():
#     print(f"NOISE_DIM {dim:2d} -> Variance: {moments['var']:.4f} | Kurtosis: {moments['kurt']:.4f}")
```

```
=====
Training 1D GAN with NOISE_DIM = 1
=====
```

```
Dim 1: 100% 800/800 [01:08<00:00, 13.13it/s]
```

```
=====
Training 1D GAN with NOISE_DIM = 5
=====
```

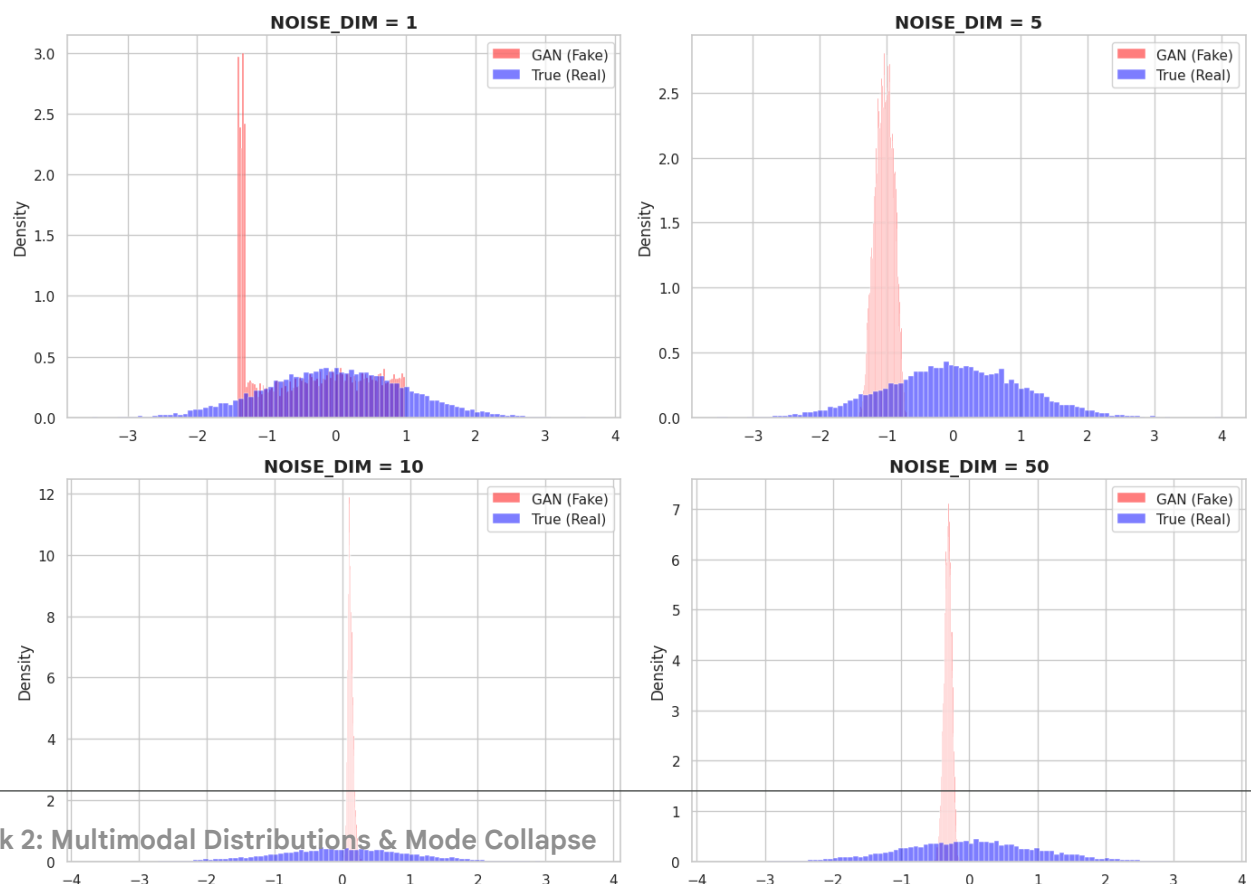
```
Dim 5: 100% 800/800 [01:08<00:00, 12.31it/s]
```

```
=====
Training 1D GAN with NOISE_DIM = 10
=====
```

```
Dim 10: 100% 800/800 [01:14<00:00, 13.03it/s]
```

```
=====
Training 1D GAN with NOISE_DIM = 50
=====
```

```
Dim 50: 100% 800/800 [01:08<00:00, 12.39it/s]
```



Task 2: Multimodal Distributions & Mode Collapse

Hypothesis: Can a vanilla BCE-loss GAN learn to generate data spanning multiple disjoint peaks (a Mixture of Gaussians)? **Analysis:** Standard GANs penalize the Generator for producing fake-looking data, but they do *not* mathematically penalize it for a lack of diversity. This reliably leads to **Mode Collapse**, where the Generator outputs only one peak and ignores the rest of the data manifold.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras.optimizers import Adam
from tqdm.notebook import tqdm

# =====
# 1. Generate 1D Mixture of Gaussians (MoG)
# =====
def generate_1d_mog(n_samples=12800):
    """Generates data with two distinct modes (peaks) at -4 and +4."""
    # Randomly assign each sample to mode 1 (-4) or mode 2 (+4)
    centers = np.random.choice([-4.0, 4.0], size=n_samples)
    # Add standard Gaussian noise to spread them out
    data = centers + np.random.randn(n_samples)
    return data.reshape(-1, 1) # Ensure shape is (n_samples, 1)

# Parameters
noise_dim = 10
epochs = 1500
```

```

batch_size = 128
x_train_mog = generate_1d_mog(n_samples=12800)

print(f"{'='*50}")
print("Training GAN on a Mixture of Gaussians (Multimodal)")
print(f"{'='*50}")

# =====
# 2. Fresh Model Initialization
# =====
# We slightly increase capacity (hidden_dim=128) to give it a fighting chance
generator_mog = set_model(noise_dim, 1, hidden_dim=128, n_layers=3, activation='linear', loss='mean_squared_error')
discriminator_mog = set_model(1, 1, hidden_dim=128, n_layers=3, activation='sigmoid', loss='binary_crossentropy')

discriminator_mog.compile(loss='binary_crossentropy', optimizer=Adam(learning_rate=0.0002, beta_1=0.5))
discriminator_mog.trainable = False
gan_mog = get_gan_network(discriminator_mog, noise_dim, generator_mog, optimizer=Adam(learning_rate=0.0002, beta_1=0.5))

d_losses_mog, g_losses_mog = [], []

# =====
# 3. Training Loop
# =====
for e in tqdm(range(1, epochs + 1), desc="Training MoG GAN"):

    # --- Train Discriminator ---
    noise = np.random.rand(batch_size, noise_dim)
    # Sample real data from our MoG distribution
    true_batch = x_train_mog[np.random.choice(x_train_mog.shape[0], batch_size, replace=False), :]
    generated_values = generator_mog.predict(noise, verbose=0)

    X = np.concatenate([true_batch, generated_values])
    y_dis = np.zeros(2 * batch_size)
    y_dis[:batch_size] = 0.9 # Label smoothing

    discriminator_mog.trainable = True
    d_loss = discriminator_mog.train_on_batch(X, y_dis)
    d_losses_mog.append(d_loss)

    # --- Train Generator ---
    discriminator_mog.trainable = False
    noise = np.random.rand(batch_size, noise_dim)
    y_gen = np.ones(batch_size)
    g_loss = gan_mog.train_on_batch(noise, y_gen)
    g_losses_mog.append(g_loss)

# =====
# 4. Evaluation and Plotting
# =====
test_noise = np.random.rand(10000, noise_dim)
final_generated_mog = generator_mog.predict(test_noise, verbose=0).flatten()
true_mog_samples = generate_1d_mog(10000).flatten()

plt.figure(figsize=(14, 5))

# Plot Distributions
plt.subplot(1, 2, 1)
sns.histplot(final_generated_mog, bins=100, color='red', alpha=0.5, stat="density", label='GAN Generated (Fake)')
sns.histplot(true_mog_samples, bins=100, color='blue', alpha=0.5, stat="density", label='True MoG (Real)')
plt.title("Distribution Comparison: GAN vs True MoG", fontweight='bold')
plt.xlabel("Value")
plt.legend()

# Plot Losses
plt.subplot(1, 2, 2)
plt.plot(d_losses_mog, label='Discriminator Loss', alpha=0.7)
plt.plot(g_losses_mog, label='Generator Loss', alpha=0.7)
plt.title("Training Losses", fontweight='bold')
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()

```

Task 3: Latent Space Tracking (MNIST Output Evolution)

Engineering Strategy: To prove the GAN is learning a deterministic mapping from $z \rightarrow x$ rather than merely memorizing outputs or outputting random static, we hold a single z latent vector completely constant. **Execution:** By passing the exact same noise matrix

through the network at specific epoch intervals, we can visually track the mathematical gradients carving the spatial relationships into the Generator's weights over time.

```
import numpy as np
import matplotlib.pyplot as plt
from tqdm.notebook import tqdm
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Input, Dense, Dropout, LeakyReLU
from tensorflow.keras.optimizers import Adam

# =====
# 0. CLEAN SLATE: Rebuild MNIST Architecture
# =====
z_dim = 100
adam_d = Adam(learning_rate=0.0002, beta_1=0.5)
adam_g = Adam(learning_rate=0.0002, beta_1=0.5)

# Rebuild Generator (100 -> 784)
g = Sequential(name="MNIST_Generator")
g.add(Input(shape=(z_dim,)))
g.add(Dense(256)); g.add(LeakyReLU(alpha=0.2))
g.add(Dense(512)); g.add(LeakyReLU(alpha=0.2))
g.add(Dense(1024)); g.add(LeakyReLU(alpha=0.2))
g.add(Dense(784, activation='sigmoid'))

# Rebuild Discriminator (784 -> 1)
d = Sequential(name="MNIST_Discriminator")
d.add(Input(shape=(784,)))
d.add(Dense(1024)); d.add(LeakyReLU(alpha=0.2)); d.add(Dropout(0.3))
d.add(Dense(512)); d.add(LeakyReLU(alpha=0.2)); d.add(Dropout(0.3))
d.add(Dense(256)); d.add(LeakyReLU(alpha=0.2)); d.add(Dropout(0.3))
d.add(Dense(1, activation='sigmoid'))

d.compile(loss='binary_crossentropy', optimizer=adam_d, metrics=['accuracy'])
d.trainable = False

# Rebuild GAN
inputs = Input(shape=(z_dim,))
gan = Model(inputs, d(g(inputs)), name="MNIST_GAN")
gan.compile(loss='binary_crossentropy', optimizer=adam_g, metrics=['accuracy'])

# =====
# 1. Initialize Fixed Latent Space
# =====
np.random.seed(42) # Lock seed for consistent tracking
fixed_noise = np.random.normal(0, 1, size=(10, z_dim))
historical_images = {}

def train_and_track(epochs=15, track_epochs=[5, 10, 15], BATCH_SIZE=128):
    batchCount = int(X_train.shape[0] / BATCH_SIZE)
    print(f"{' '*50}")
    print(f"Starting Training. Snapshots will be taken at Epochs: {track_epochs}")
    print(f"{' '*50}")

    for e in range(1, epochs + 1):
        for _ in tqdm(range(batchCount), leave=False, desc=f"Epoch {e}"):

            # --- Train Discriminator ---
            image_batch = X_train[np.random.randint(0, X_train.shape[0], size=BATCH_SIZE)]
            noise = np.random.normal(0, 1, size=(BATCH_SIZE, z_dim))
            generated_images = g.predict(noise, verbose=0)

            X = np.concatenate((image_batch, generated_images))
            y_real = np.ones(BATCH_SIZE) * 0.9 # Label smoothing
            y_fake = np.zeros(BATCH_SIZE)
            y_dis = np.concatenate((y_real, y_fake))

            d.trainable = True
            d_loss = d.train_on_batch(X, y_dis)

            # --- Train Generator ---
            noise = np.random.normal(0, 1, size=(BATCH_SIZE, z_dim))
            y_gen = np.ones(BATCH_SIZE)

            d.trainable = False
            g_loss = gan.train_on_batch(noise, y_gen)

            # =====
            # 2. The Tracking Mechanism
            # =====
            if e in track_epochs:
                # Save generated images for visualization
```

```

gen_imgs = g.predict(fixed_noise, verbose=0)
historical_images[e] = gen_imgs.reshape(10, 28, 28)
print(f"-> Snapshot saved for Epoch {e}")

return historical_images

# Execute the training
tracked_data = train_and_track(epochs=15, track_epochs=[5, 10, 15])

# =====
# 3. Visualization: The Evolutionary Grid
# =====
def plot_evolution(history_dict):
    epochs_recorded = list(history_dict.keys())
    n_epochs = len(epochs_recorded)
    n_images = 10

    plt.figure(figsize=(15, 2.5 * n_epochs))

    for row, epoch in enumerate(epochs_recorded):
        images_to_plot = history_dict[epoch]

        for col in range(n_images):
            plot_idx = row * n_images + col + 1
            ax = plt.subplot(n_epochs, n_images, plot_idx)

            plt.imshow(images_to_plot[col], cmap='gray_r', interpolation='nearest')
            plt.axis('off')

            if col == 0:
                ax.set_title(f"Epoch {epoch}", fontweight='bold', fontsize=14, pad=10, loc='left')

    plt.suptitle("Generator Evolution: Fixed Latent Vectors Over Time", fontsize=18, y=0.98, fontweight='bold')
    plt.tight_layout(rect=[0, 0.03, 1, 0.95])
    plt.show()

plot_evolution(tracked_data)

```

```

=====
Starting Training. Snapshots will be taken at Epochs: [5, 10, 15]
=====
/usr/local/lib/python3.12/dist-packages/keras/src/layers/activations/leaky_relu.py:41: UserWarning: Argument `alpha` is de
warnings.warn(

```

-> Snapshot saved for Epoch 5

-> Snapshot saved for Epoch 10

-> Snapshot saved for Epoch 15

Generator Evolution: Fixed Latent Vectors Over Time

Epoch 5



Epoch 10



Start coding or generate with AI.

Task 4: Gradient Starvation (Label Smoothing Ablation)

Hypothesis: How does absolute discriminator certainty impact Generator optimization? Analysis: Setting real labels to exactly 1.0 forces the discriminator into high confidence. In a sigmoid activation, pushing outputs exactly to 1.0 causes the derivative to approach zero ($\nabla_y \rightarrow 0$). Without gradients, the generator receives zero learning signal. Smoothing labels to 0.9 ensures continuous, non-zero gradient flow, preventing convergence failure.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras.optimizers import Adam
from tqdm.notebook import tqdm

# Set random seeds to ensure we are comparing apples to apples
np.random.seed(100)

epochs_per_test = 1000
batch_size = 128
noise_dim = 5 # Using 5 as a stable baseline from our previous experiment
x_train = generate_data(n_samples=12800, n_dim=1)

label_configs = {
    "Without Smoothing (Labels = 1.0)": 1.0,
    "With Smoothing (Labels = 0.9)": 0.9
}

results = {}

for label_name, real_label_value in label_configs.items():
    print(f"\n{' '*50}")
    print(f"Training GAN {label_name}")
    print(f"{' '*50}")

    # 1. Fresh Initialization
    generator = set_model(noise_dim, 1, n_layers=2, activation='linear', loss='mean_squared_error')
    discriminator = set_model(1, 1, n_layers=2, activation='sigmoid', loss='binary_crossentropy')

    discriminator.compile(loss='binary_crossentropy', optimizer=Adam(learning_rate=0.001))
    discriminator.trainable = False
    gan = get_gan_network(discriminator, noise_dim, generator, optimizer=Adam(learning_rate=0.001))

    d_losses, g_losses = [], []

    # 2. Training Loop
    for e in tqdm(range(1, epochs_per_test + 1), desc=label_name):
        # Train Discriminator
        noise = np.random.rand(batch_size, noise_dim)
        true_batch = x_train[np.random.choice(x_train.shape[0], batch_size, replace=False), :]
        generated_values = generator.predict(noise, verbose=0)
        X = np.concatenate([true_batch, generated_values])

        y_dis = np.zeros(2 * batch_size)
        y_dis[:batch_size] = real_label_value # <--- THE CRITICAL CHANGE IS HERE

        discriminator.trainable = True
        d_loss = discriminator.train_on_batch(X, y_dis)
        d_losses.append(d_loss)

        # Train Generator
        discriminator.trainable = False
        noise = np.random.rand(batch_size, noise_dim)
        y_gen = np.ones(batch_size) # G always wants D to output 1.0
        g_loss = gan.train_on_batch(noise, y_gen)
        g_losses.append(g_loss)

    # 3. Evaluate
    test_noise = np.random.rand(10000, noise_dim)
    final_generated = generator.predict(test_noise, verbose=0).flatten()

    results[label_name] = {
        'generated': final_generated,
        'd_losses': d_losses,
        'g_losses': g_losses
    }

# =====
```

```
# 4. Plotting the Instability
# =====
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
true_gaussian = [np.random.randn() for _ in range(10000)]

for idx, (label_name, data) in enumerate(results.items()):
    # Plot Distributions
    ax_dist = axes[0, idx]
    sns.histplot(data['generated'], bins=100, color='red', alpha=0.5, stat="density", label='GAN (Fake)', ax=ax_dist)
    sns.histplot(true_gaussian, bins=100, color='blue', alpha=0.5, stat="density", label='True (Real)', ax=ax_dist)
    ax_dist.set_title(f"Distribution: {label_name}", fontsize=14, fontweight='bold')
    ax_dist.legend()

    # Plot Losses
    ax_loss = axes[1, idx]
    ax_loss.plot(data['d_losses'], label='Discriminator Loss', alpha=0.8, color='blue')
    ax_loss.plot(data['g_losses'], label='Generator Loss', alpha=0.8, color='orange')
    ax_loss.set_title(f"Loss Curves: {label_name}", fontsize=14, fontweight='bold')
    ax_loss.set_xlabel("Epochs")
    ax_loss.set_ylabel("Loss")
    ax_loss.legend()

plt.tight_layout()
plt.show()
```

```
=====
Training GAN Without Smoothing (Labels = 1.0)
=====
```

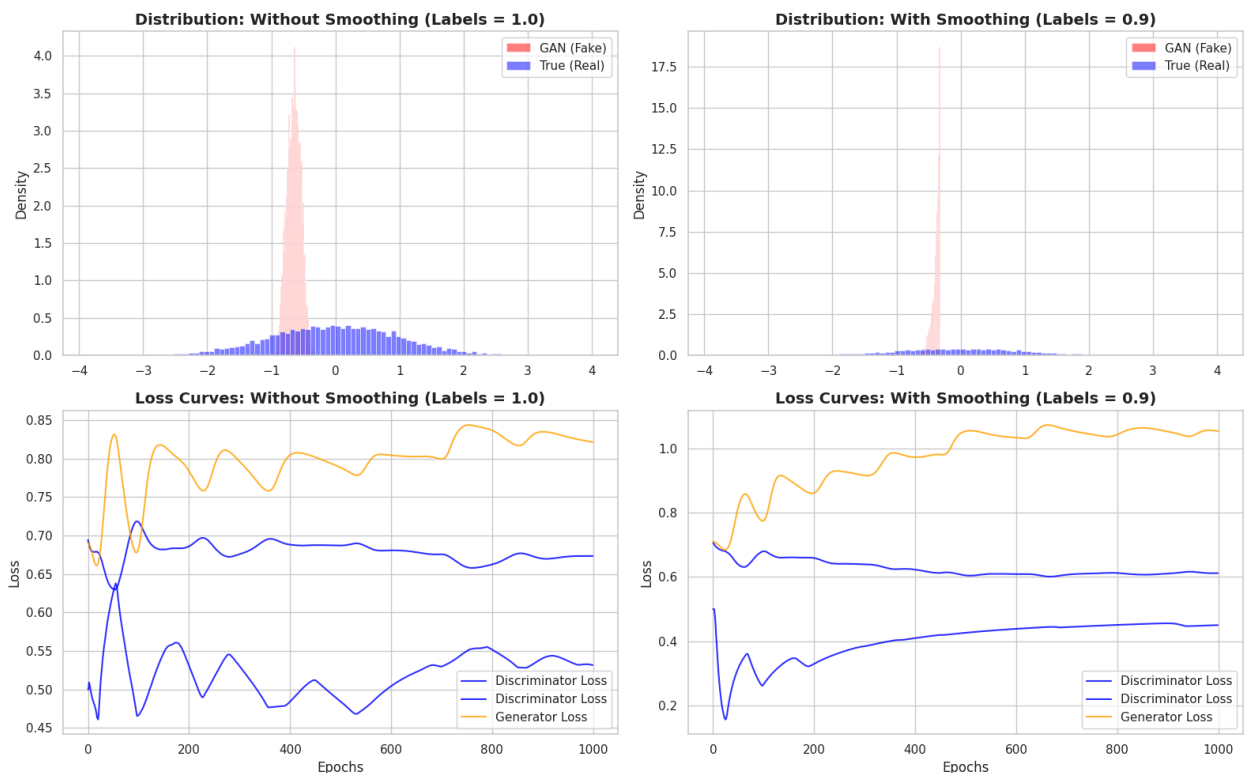
Without Smoothing (Labels = 1.0): 100%

1000/1000 [01:28<00:00, 12.73it/s]

```
=====
Training GAN With Smoothing (Labels = 0.9)
=====
```

With Smoothing (Labels = 0.9): 100%

1000/1000 [01:21<00:00, 12.07it/s]



Task 5: Capacity Imbalance (Shallow Generator vs. Deep Discriminator)

Hypothesis: What happens to the Nash Equilibrium if the Discriminator has vastly superior representational capacity? **Analysis:** An overpowered Discriminator acts as a "brick wall." It achieves perfect classification almost instantly. Because the Generator relies entirely on the Discriminator's errors to calculate its updates, a perfect Discriminator yields zero useful gradients. The Generator gets stuck in a local minimum, unable to bend its limited parameters to match the target variance.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras.optimizers import Adam
from tqdm.notebook import tqdm

# =====
# 1. Setup Data and Unbalanced Architectures
# =====
noise_dim = 10
epochs = 1000
batch_size = 128
x_train_unbalanced = generate_data(n_samples=12800, n_dim=1)

print(f"{' '*60}")
print("Experiment: Shallow Generator vs. Deep Discriminator")
print(f"{' '*60}")

# SHALLOW GENERATOR: 1 Hidden Layer, 16 neurons
generator_shallow = set_model(noise_dim, 1, hidden_dim=16, n_layers=1, activation='linear', loss='mean_squared_error')

# DEEP DISCRIMINATOR: 5 Hidden Layers, 128 neurons
discriminator_deep = set_model(1, 1, hidden_dim=128, n_layers=5, activation='sigmoid', loss='binary_crossentropy')

# Compile models
discriminator_deep.compile(loss='binary_crossentropy', optimizer=Adam(learning_rate=0.001))
discriminator_deep.trainable = False
gan_unbalanced = get_gan_network(discriminator_deep, noise_dim, generator_shallow, optimizer=Adam(learning_rate=0.001))

d_losses_unb, g_losses_unb = [], []

# =====
# 2. Training Loop
# =====
for e in tqdm(range(1, epochs + 1), desc="Training Unbalanced GAN"):

    # --- Train Deep Discriminator ---
    noise = np.random.rand(batch_size, noise_dim)
    true_batch = x_train_unbalanced[np.random.choice(x_train_unbalanced.shape[0], batch_size, replace=False), :]
    generated_values = generator_shallow.predict(noise, verbose=0)

    X = np.concatenate([true_batch, generated_values])
    y_dis = np.zeros(2 * batch_size)
    y_dis[:batch_size] = 0.9 # Keeping label smoothing to isolate architecture effects

    discriminator_deep.trainable = True
    d_loss = discriminator_deep.train_on_batch(X, y_dis)
    d_losses_unb.append(d_loss)

    # --- Train Shallow Generator ---
    discriminator_deep.trainable = False
    noise = np.random.rand(batch_size, noise_dim)
    y_gen = np.ones(batch_size)
    g_loss = gan_unbalanced.train_on_batch(noise, y_gen)
    g_losses_unb.append(g_loss)
```